Norwegian University of Science and Technology
Department of Mathematical Sciences

TMA4215 - Project part 1
Deadline September 29th, 2025

Practical information

- This part of the project counts for 20% of the final mark. The work should be performed and handed in individually. There will be a second part of the project in November counting for another 10% of the final mark.
- A major part of this subproject will be the implementation (and testing) of codes in Python. You will submit a report in form of a Jupiter notebook file. This file will contain the code as well as explanations and comments in text of your findings and of your numerical results. Make sure that your code contains a meaningful documentation, in particular a reasonable number of comments to explain what you have done. Also test the programs before submission and comment on all strange behaviour of the programs in your report.
- When you present numerical results, these should be reproducible. This means that you will have to provide all relevant parameters.
- The Jupiter notebook report and the code should be submitted in Blackboard.
- There will be five exercise sessions devoted to the supervision of the project (September 12, 15, 19, 22 and 26) and in addition no lectures on the 16th, 19th and 23rd of September to allow time for the project. Meet the supervisor in room 634, send him an email for an appointment. You are welcome to ask questions to the lecturer about the project, please send an e-mail to agree on a time if you want to be sure that we are available.
- If you find mistakes please notify to the lecturer and TA.

Approximation of functions

Approximation theory is at the core of applied mathematics. Many tasks in scientific computing and data analysis are hinging on the use of methods for the approximation of functions in one or in several variables. For example, numerical methods for quadrature, for the numerical solution of ordinary and partial differential equations are founded on approximation methods.

In this project the goal is to gain experience with different simple classical techniques for the approximation of functions.

You will have to implement different numerical methods for the interpolation of real valued functions defined on an interval $[a, b]$ of the real line.

Univariate interpolation

Problem 1

We consider a continuous function $f : [a, b] \to \mathbb{R}$ which we want to approximate with different methods. In what follows, we will always denote with $n + 1$ the number of nodes of interpolation for an interpolation polynomial or some other interpolating function.

We will sometimes use a finer grid of points with $N >> n + 1$ points which will be used to plot the results or to approximate function norms (such as the ∞-norm and the 2-norm of functions) or for other purposes.

a) Implement the Lagrange interpolation polynomial for an arbitrary set of distinct nodes $x_0, \ldots, x_n$ and for values $y_0, \ldots, y_n$. Test you code on equidistant nodes and on Chebishev nodes considering first a function $f \in C^\infty$ defined on the interval $[-1, 1]$. Then generalise the method based on Chebishev nodes transforming the nodes to an arbitrary interval $[a, b]$ by applying the one-to-one transformation $\Psi : [-1, 1] \to [a, b]$, $\Psi(x) = \frac{b-a}{2}x + \frac{b+a}{2}$.

Write your code as a Python function.

Input:

- The data that we want to interpolate: x_i and y_i, $i = 0, \ldots, n$.
- The value of x where we want to evaluate the interpolation polynomial, x can be an array.

Output: the value of the interpolation polynomial in x.

Your answer to this point should include a plot of the interpolation of the Runge function $f(x) = 1/(x^2 + 1)$ with equidistant nodes and Chebishev nodes on the interval $[-5, 5]$ with $n = 10$.

b) Interpolation on both equidistant nodes and Chebishev nodes should converge in both the max-norm and 2-norm for the following functions:

- $f(x) = \cos(2\pi x)$, $x \in [0, 1]$;
- $f(x) = e^{3x} \sin(2x)$, $x \in [0, \pi/4]$.

Can you explain why?

Of course on the interpolation nodes the error is zero, however on other points of the interval $[a, b]$ the error might be small but is not zero. In order to check convergence with a numerical experiment, we want to estimate $\|f - p_n\|$, where $\|\cdot\|$ is the max-norm or the 2-norm for continuous functions, and for different values of n.

Consider a grid which has 100 times as many points as the interpolation polynomial: $x_0 = \eta_0 < \eta_1 < \cdots < \eta_N = x_n$ with N "large", e.g. $N = 100n_{\max}$, where $n_{\max}$ is the largest degree of the interpolation polynomial that you consider in your numerical experiments. Compute the following approximations of the max-norm and of the 2-norm respectively

$$\|f - p_n\|_\infty \approx \max_{\eta_0,\ldots,\eta_N} |f(\eta_i) - p_n(\eta_i)|,$$

$$\|f - p_n\|_2 \approx \frac{\sqrt{b-a}}{\sqrt{N}} \left(\sum_{i=0}^{N} (f(\eta_i) - p_n(\eta_i))^2 \right)^{\frac{1}{2}}.$$

Make a plot of the estimated error $\|p_n - f\|$ as a function of n to check convergence. Use the approximations of $\|\cdot\|_\infty$ and $\|\cdot\|_2$ given above[1].

Consider $f(x) = \cos(2\pi x)$, $x \in [0,1]$. Can you use the well known error bound for the interpolation polynomial, derive a bound for the max-norm of each of the derivatives of f on $[0,1]$, and derive an error bound for the interpolation error valid for all n for this particular function? Plot the error bound as a function of n and compare with the numerical estimates of the norm of the error that you have computed earlier.

Your answer should include two plots one for each of the two functions with the estimated error in the max and 2-norm, one of the plots should include also the the derived error bound as a function of n.

c) Piecewise-polynomial approximation

Suppose f is continuous and is the function you want to approximate, here we want to obtain a piece-wise polynomial and continuous approximation of f on $[a,b]$.

Subdivide the interval $[a,b]$ in the disjoint union of K subintervals $a = v_0 < v_1 < \cdots < v_K = b$. Implement now a method that performs Lagrangian interpolation on $n+1$ nodes on each subinterval $[v_i, v_{i+1}]$. Use equidistant nodes on the subintervals. Fix $n = 1, 2, \ldots, 10$. Make a plot of the interpolation error in the max-norm as a function of K, give numerical evidence that the method converges as $K \to \infty$.

If you have implemented the interpolation polynomial in part **a)** and **b)** *wisely*, you should be able to reuse that function here for the interpolation on each subinterval. Continuity will arise naturally by including the endpoints v_i and v_{i+1} as nodes of the interpolation on the interval $[v_i, v_{i+1}]$.

Discuss comparison of the results with the polynomial interpolation of the previous question. Use the same test function for comparison. Produce a plot of the error in ∞-norm as a function of the number of discretization

[1] The weigthed sum in the approximation of the 2-norm is a quadrature formula applied to approximate the integral in $\|g\|_2 = (\int_a^b (g(x))^2\,dx)^{\frac{1}{2}}$. We will study quadrature formulae later in this course.

nodes. Comment briefly on what are the advantages and the disadvantages of this and of the previous methods. Which one would you recommend to use in which situation? Discuss computational cost for the various methods.

d) We will now consider a completely different method for approximating f:

$$f(x) \approx \tilde{f}(x) = \sum_{i=0}^{n} w_i \, \phi(|x - x_i|), \quad \phi(r) = \exp(-(\varepsilon r)^2),$$

this is an example of a radial basis function (RBF)[2], and ε is the shape parameter. The basis functions $\phi(|x - x_i|)$ are equal to 1 in $x = x_i$ and decay to zero as the distance $|x - x_i|$ increases; the size of the shape parameter ε determines the speed of decay of $\phi(|x - x_i|)$ to zero as $|x - x_i|$ increases.

Imposing interpolation conditions

$$\tilde{f}(x_i) = f(x_i), \quad i = 0, \ldots, n,$$

one finds that the values $\mathbf{w} = [w_0, \ldots, w_n]^T$ are obtained solving the linear system

$$M\mathbf{w} = \mathbf{f},$$

where $\mathbf{f} := [f(x_0), \ldots, f(x_n)]^T$, and M is an $(n+1) \times (n+1)$ matrix with entries

$$M_{i,j} := \phi(|x_i - x_j|),$$

so the weights $\mathbf{w}$ are functions of the nodes $\mathbf{x}$.

Implement an RBF function with

Input:

- The data that we want to interpolate: x_i and y_i, $i = 0, \ldots, n$.
- The value of x where we want to evaluate the RBF, x can be an array.

Output: the value of the interpolating RBF in x.

Test the RBF implementation on the Runge function on $[-5, 5]$ ($n = 10$) and another of the functions considered in the previous exercises. Choose ε by trial and error so to obtain a good approximation of the functions considered.

e) **Optimal distribution of the nodes**. Assume now that the function f is known on a fine grid of $N + 1$ points, $\eta_0, \ldots, \eta_N$ in the interval $[a, b]$ including the endpoints, e.g. equidistant points and with a large value of N, say $N = 1000$.

[2] RBFs are strictly related to the so called Gaussian Processes in Statistics.

Fixed $n+1$ the number of interpolation nodes (with $n << N-1$), and fixed the function f that we want to interpolate, we seek an optimal distribution of the $n+1$ interpolation nodes $x_0, \ldots, x_n$ on $[a, b]$ and an optimal shape parameter ϵ, minimising the error in the approximate 2-norm (same as earlier considered in exercise **b**). To this purpose we solve the following optimization problem

$$\min_{x_0,\ldots,x_n \in [a,b],\varepsilon} \frac{b-a}{N} \sum_{k=0}^{N} (f(\eta_k) - \tilde{f}(\eta_k))^2 \quad \text{subject to} \quad \tilde{f}(x_j) = f(x_j),\ j = 0, \ldots, n.$$

This is a *least squares* problem. You should solve the problem with an optimization algorithm based on gradient descent, see the appendix for a quick review of gradient descent. Here $\tilde{f}$ is a RBF and depends on $x_0, \ldots, x_n$ through the interpolation conditions, $\tilde{f}(x_j) = f(x_j)$, $j = 0, \ldots, n$.

Let $\mathbf{x} = [x_0, \ldots, x_n]$ be a vector containing the nodes of the interpolation. Here the cost function that we want to mimimizze is $\mathcal{C} : \mathbb{R}^{n+1} \to \mathbb{R}$,

$$\mathcal{C}([\mathbf{x}, \varepsilon]) := \frac{b-a}{N} \sum_{k=0}^{N} (f(\eta_k) - \tilde{f}(\eta_k))^2, \qquad \tilde{f}(\eta) = \sum_{i=0}^{n} w_i \phi(|\eta - x_i|), \quad (1)$$

$$\mathbf{w} = M^{-1}\mathbf{f}, \qquad \mathbf{w} = [w_0, \ldots, w_n].$$

where $c_k := f(\eta_k)$ are known values, $\tilde{f}$ is the RBF. So the cost function depends on $\mathbf{x}$ and ε through $\tilde{f}$. Use gradient descent to optimize over the location of the nodes and the shape parameter ε. Initialize the gradient descent iteration with equidistant nodes: $x_j = a + j(b-a)/n$, $j = 0, \ldots, n$.

How to proceed:

- You need to write a function in Python implementing the cost function (1) taking a set of nodes $\mathbf{x}$ and ε in input and returning the value $\mathcal{C}([\mathbf{x}, \varepsilon])$.
- You can then use `autograd` in Python to compute the gradient $\nabla\mathcal{C}$.
- It is convenient that this function takes in input only $[\mathbf{x}, \varepsilon]$, because then the automatic differentiation routine will compute the gradient with respect to $[\mathbf{x}, \varepsilon]$ and nothing else. However, $\mathcal{C}$ will need to have access to the values $c_k = f(\eta_k)$, $k = 0, \ldots, N$, the data that you generate at the start of your experiments. You will have to make sure that the function you write in Python to implement $\mathcal{C}([\mathbf{x}, \varepsilon])$ has access to the correct data and to the necessary information.
- Finally you will implement the gradient descent iteration with a stopping criterion using the computed gradient. Use also a maximum number of iterations to prevent your code to run for a very long time.

Plot the convergence history of the gradient descent iteration to see that indeed the cost function decreases as the number of iterations increases.

Compare the results you obtain against the exact value of the function, and against the interpolation methods of the previous point: compute the optimal nodes for different, increasing values of n, then compare the 2-norm of the error with the error of the interpolation on equidistant nodes and on Chebishev nodes as functions of n. You can make a table with the results. Comment the results you obtain.

f) Suppose that in the previous exercise $\tilde{f}$ is replaced by the interpolation polynomial p_n and that you seek optimal nodes $\mathbf{x} = [x_0, \ldots, x_n]$ such that the cost function

$$\mathcal{C}(\mathbf{x}) := \frac{b-a}{N} \sum_{k=0}^{N} (f(\eta_k) - p_n(\eta_k))^2, \quad p_n(\eta) = \sum_{i=0}^{n} \ell_i(\eta) f(x_i), \tag{2}$$

is minimized. Here

$$\ell_i(\eta) = \prod_{j=0, j\neq i}^{n} \frac{\eta - x_j}{x_i - x_j}$$

are the Lagrange basis functions. What is the relation between the *best approximation polynomial of degree* n and the interpolation polynomial $p_n(x)$ on this set of nodes? Could you suggest an alternative way of computing p_n? (You do not need to implement anything here only give an answer to the question).

APPENDIX

Gradient Descent

Assume we want to minimise the function $\mathcal{C} : \mathbb{R}^m \to \mathbb{R}$ which we assume to be continuously differentiable. A necessary condition for $\mathbf{x}^* \in \mathbb{R}^m$ to be a minimiser of the function is that

$$\nabla \mathcal{C}(\mathbf{x}^*) = 0,$$

where $\nabla \mathcal{C}$ is the gradient of $\mathcal{C}$. An algorithm for finding mimima of $\mathcal{C}$ can be therefore obtained simply searching for zeros of the function $\nabla \mathcal{C}$.

The gradient descent algorithm is an iterative method to approximate the solution of $\nabla \mathcal{C} = 0$ of the following form

$$\mathbf{x}^{k+1} = \mathbf{x}^k - \tau \nabla \mathcal{C}(\mathbf{x}^k), \quad k = 0, 1, 2, \ldots$$

where $\tau > 0$ is a step-size parameter (learning rate) which can be chosen adaptively (i.e. changed at every step). The iteration is stopped when a stopping criterion is met, e.g. when $\|\mathbf{x}^k - \mathbf{x}^{k-1}\| \leq TOL$ where $TOL > 0$ is a specified tolerance, an alternative stopping criterion is $\|\nabla \mathcal{C}(\mathbf{x}^k)\| \leq TOL$. Use small values of τ, e.g. $\tau = 0.1$ or $\tau = 0.01$. To improve the performance of the gradient descent you might consider using a *backtracking* strategy, i.e. a simple strategy

Algorithm 1 Gradient descent with backtracking.

Input: initial guess for nodes of interpolation $\mathbf{x}$ and parameter L, hyperparameters $\overline{\rho} > 1$ and $\underline{\rho} < 1$.

```
compute gradient ∇C of the cost function C using autograd
compute φ = C(x)
for k = 1, ... do
    evaluate the gradient in x: g = ∇C(x)
    for t = 1, ... do
        update: x̃ = x − (1/L) g
        compute φ̃ = C(x̃)
        if φ̃ ≤ φ + ⟨g, x̃ − x⟩ + (L/2)‖x̃ − x‖² then
            accept: x = x̃ , φ = φ̃, L = ρ̲L
            break inner loop
        else reject: L = ρ̄L
```

to select τ at each step that guarantees that the cost function is reduced at every iteration.

Backtraking for gradient descent

This is a strategy that allows you to chose the step-size of the gradient descent adaptively at each iteration and in such a way that the cost function is guaranteed to decrease of a certain amount at every step.

Use equidistant nodes as initial $\mathbf{x}$, use $L = 1, 10, 100$ as initial L.